

FindYourBook

Relazione di Progetto - Ingegneria del Software

ISPW PROJECT - University of Rome
Tor Vergata.
AURORA CICCHETTA
2025-2026

Table of contents

1. SOFTWARE REQUIREMENT SPECIFICATION (SRS)	3
1.1 INTRODUCTION	3
1.1.1 Aim of the Document	3
1.1.2 Overview of the Defined System	3
1.1.3 Hardware and Software Requirements	3
1.1.4 Related Systems - Pros and Cons	4
1.2 USER STORIES	5
1.3 FUNCTIONAL REQUIREMENTS	5
1.4 USE CASES (FOCUS ON CORE FEATURES)	6
1.4.1 Overview Diagram	6
1.4.2 Internal Steps	7
2. STORYBOARDS	8
3. DESIGN	11
3.1 CLASS DIAGRAM	11
3.1.1 VOPC analysis	11
3.1.2 Design-level Diagram	12
3.2 DESIGN PATTERNS	14
1. SINGLETON	14
2. OBSERVER	14
3. STATE	14
3.3 ACTIVITY DIAGRAM	15
3.4 SEQUENCE DIAGRAM	16
3.5 STATE DIAGRAM	17
4. TESTING	18
5. CODE	19
6. VIDEO	19

1. Software Requirement Specification (SRS)

1.1 Introduction

1.1.1 Aim of the Document

The purpose of this document is to present the fundamental and specific requirements of the "FindYourBook" system, a project developed for the Software Engineering course, Academic Year 2025-2026. The document outlines the system's objectives, core functionalities, and architectural design choices to guide and document the development process.

1.1.2 Overview of the Defined System

The system is a Java-based software platform structured according to the MVC architecture, utilizing Maven for dependency management, and featuring graphical user interfaces implemented via JavaFX alongside support components and robust logging (AppLogger). The system supports the modern and efficient management of a literary catalog and personal user libraries, including the following key functionalities:

- Management of distinct user roles: Readers and Publishing Houses (*Publisher*).
- Browsing and searching within the official catalog of published books.
- Managing personal library states (*To Read*, *Reading*, *Read*) and rating books.
- Automatic tracking of reading progress and automated background or event-driven checks on reading duration.
- Real-time email notifications for registration confirmation, book publications, reading goals reached, and inactivity reminders (via SendGrid API integration).

1.1.3 Hardware and Software Requirements

- Software requirements:
 - Java 21+ (compatible with OpenJFX 23+)
 - Maven 3.8+
 - MySQL Server 8.0+ (or in-memory/JSON fallback persistence for demo mode)
 - JavaFX 23+
 - IDE: IntelliJ IDEA (recommended) or Eclipse
- Hardware requirements:
 - CPU: Dual-Core 2.0 GHz
 - RAM: 4 GB (2 GB minimum)
 - Disk Space: 1 GB
 - Operating System: Windows, MacOS, Linux (compatible with Java 21+)

1.1.4 Related Systems - Pros and Cons

FindYourBook

- PRO:
 - Free of charge and completely open access for all users.
 - Targeted specifically at structured dual user roles, bridging readers and publishing houses directly.
 - Real-time email notifications for registration confirmation, published book updates, reading goal achievements, and inactivity reminders (via SendGrid API integration).
- CONS:
 - Desktop-only software; no native mobile application available yet.
 - Inactivity checks and reading reminders are triggered locally during user login rather than synchronized with external cloud calendars.

Goodreads

- PRO:
 - Massive, global community of readers offering extensive reviews, ratings, and social interactions.
 - Cross-platform availability with robust mobile applications and web access.
 - Offers comprehensive personalized book recommendations based on user reading history and community ratings.
- CONS:
 - Heavy commercialization and advertising presence, lacking direct integration for independent publishing houses to manage catalogs and sales metrics.
 - Interface and recommendation algorithms can sometimes feel overwhelming or overly cluttered for users seeking a simple personal library manager.

1.2 User Stories

1. As a reader, I want to save a new book into my personal library under the "To Read" section, so that I can track my future reading list directly from the application.
2. As a publisher, I want to add a new title to the platform's official catalog, so that it becomes immediately visible and accessible to all readers.
3. As a reader, I want to select and search books by literary genre, so that I can choose my next reading material based on my current interests.

1.3 Functional Requirements

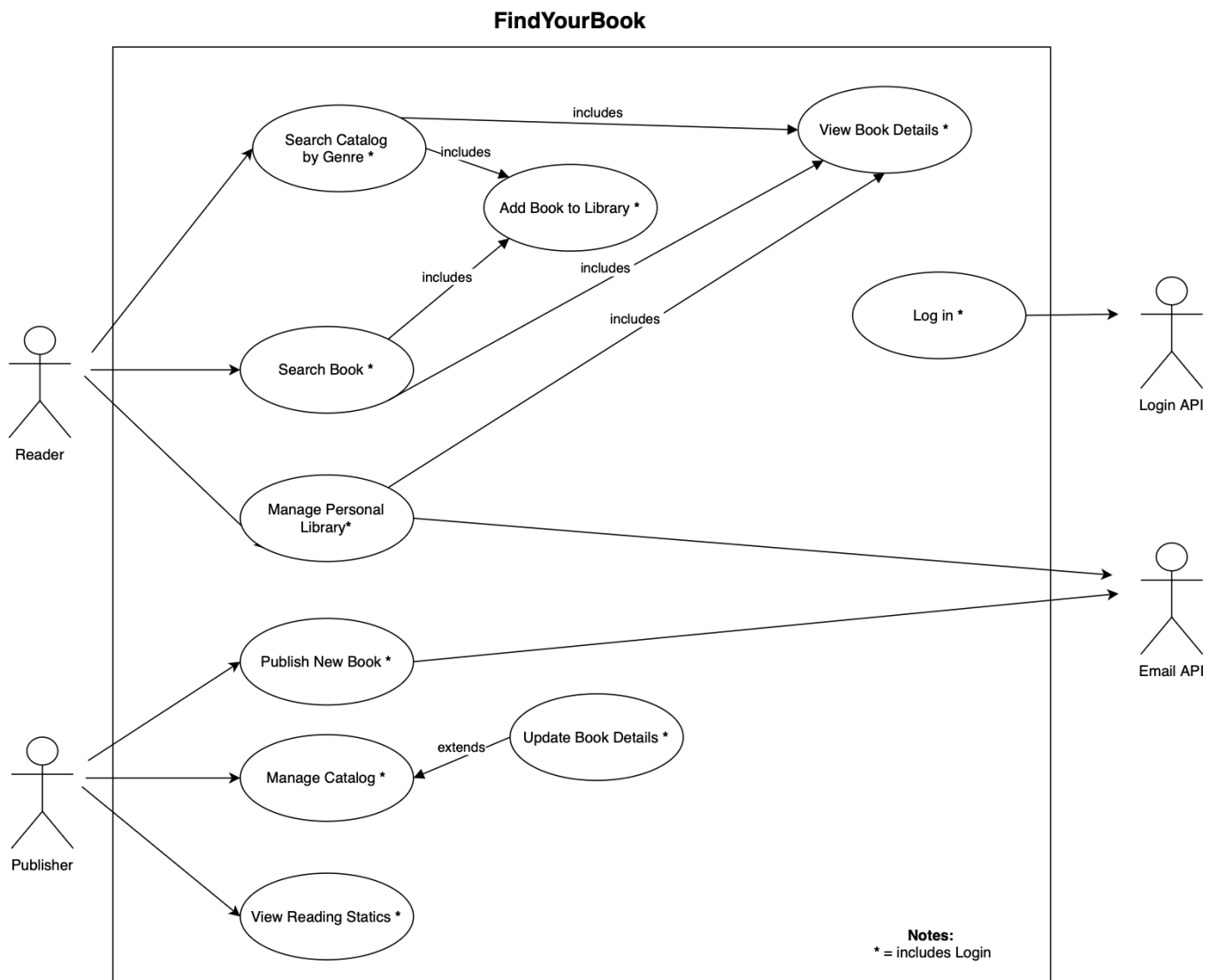
1. The system shall provide the Reader with the ability to categorize books into specific reading statuses (*To Read*, *Reading*, *Read*).
2. The system shall provide Publishing Houses with the ability to submit new book records containing the title, author, genre, description, and image URL into the central database.
3. The system shall provide publishing houses with the ability to update the details of books they have previously submitted to the central catalog.

Glossary

- Reader = A registered user of the platform whose primary role is to browse the catalog, manage personal library states, and track reading progress.
- Publishing House = A registered user or administrative entity responsible for submitting, updating, and removing book titles within the official system catalog.
- Catalog = The central repository of published books available for discovery and inclusion in personal user libraries.
- Reading Status = The specific categorization state assigned to a book within a personal library, indicating whether it is marked as *To Read*, *Reading*, or *Read*.

1.4 Use Cases (Focus on Core Features)

1.4.1 Overview Diagram



1.4.2 Internal Steps

Use case: Manage Personal Library

Main flow:

1. The reader accesses their personal account (via UC Login).
2. The reader selects the personal library function from the dashboard menu.
3. The system retrieves and displays the count of total read works associated with the reader's profile.
4. The system verifies whether any books currently in the READING state have a start date older than 30 days.
5. The system displays the user interface with the tabs to select the reading status filters (TO_READ, READING, READ).
6. The reader selects a specific reading status filter tab (TO_READ, READING, or READ).
7. The system requests and loads the book collection filtered by the selected reading status. (TO_READ, READING, or READ).
8. The system displays the grid of book cards containing titles, authors, ratings, and current statuses.
9. The reader selects a specific book card and perform one of the following action:
 - The Reader updates the reading status (TO_READ, READING, READ).
 - The Reader rates the book.
 - The Reader removes the book from the personal library.
10. The system persists the change via the database.
11. The system refreshes the library view.

Extensions :

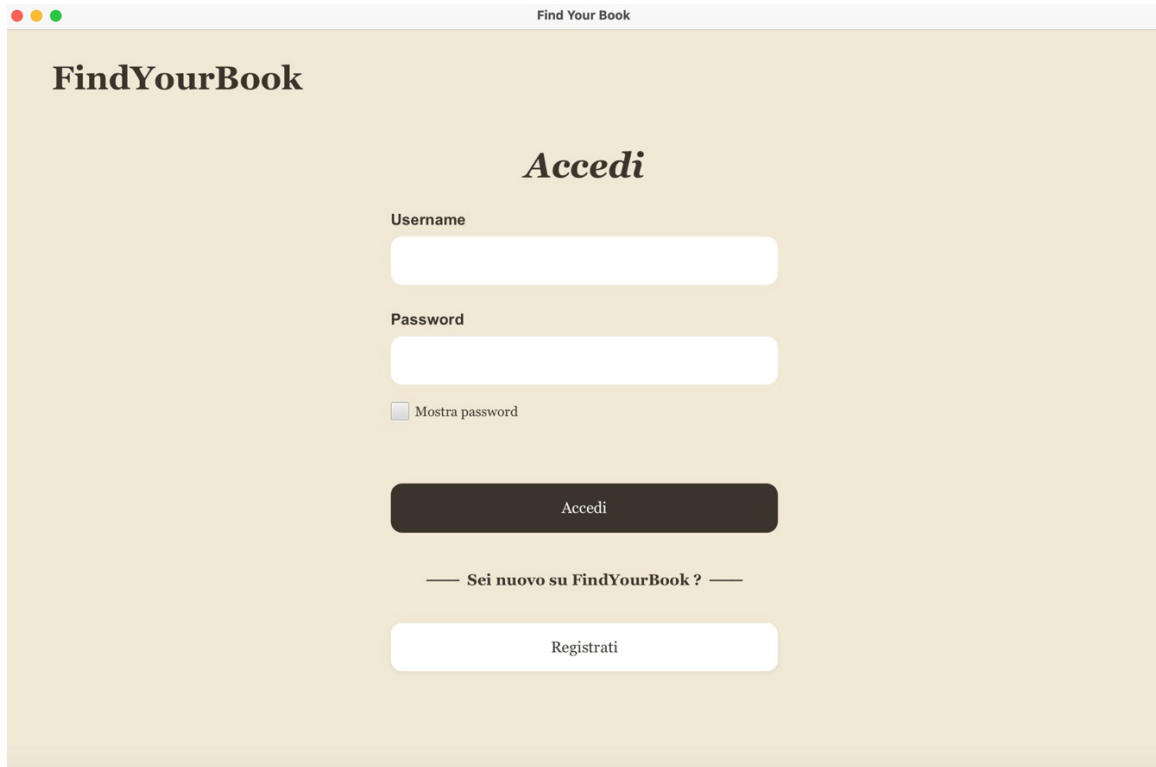
4a. Inactive reading detected: The system identifies books in the READING state started over 30 days ago and invokes the Email API (SendGrid) to send a reading inactivity reminder to the Reader's email address, then continues the main flow from step 4.

7a. No books found for the selected filter: The system displays a message indicating that no books are available in the selected section, then returns to the main flow from step 5.

9b. Completion goal reached: The Reader updates the book status specifically to READ; the system invokes the Email API to send a completion goal notification to the Reader's email address, then continues to step 10.

2. Storyboards

Login



The login form is titled "FindYourBook" and "Accedi". It features two input fields for "Username" and "Password". Below the password field is a checkbox labeled "Mostra password". A dark button labeled "Accedi" is positioned below the inputs. A link "Sei nuovo su FindYourBook ?" is centered below the button, followed by a light button labeled "Registrati".

Find Your Book

FindYourBook

Accedi

Username

Password

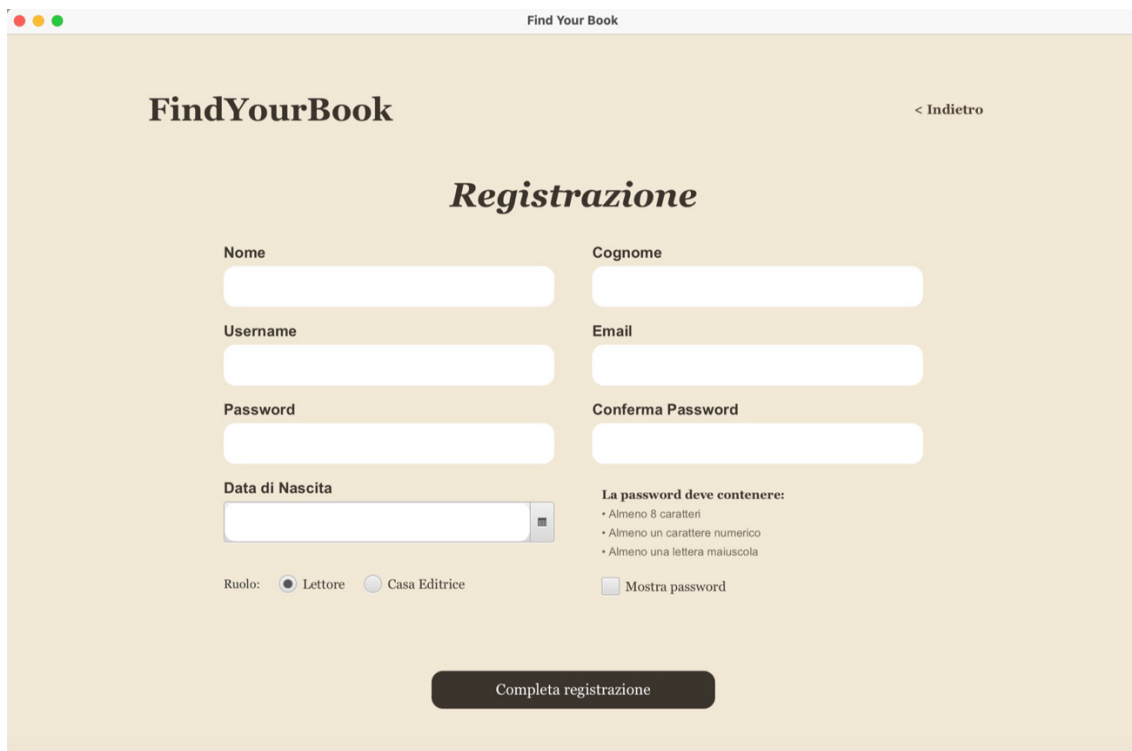
☐ Mostra password

Accedi

— Sei nuovo su FindYourBook ? —

Registrati

Registration



The registration form is titled "FindYourBook" and "Registrazione". It includes a "< Indietro" link in the top right. The form has two columns of input fields: "Nome", "Cognome", "Username", "Email", "Password", and "Conferma Password". Below the "Data di Nascita" field is a "Ruolo:" section with radio buttons for "Lettore" and "Casa Editrice". A "Mostra password" checkbox is located below the password fields. A list of password requirements is provided: "La password deve contenere:" followed by "• Almeno 8 caratteri", "• Almeno un carattere numerico", and "• Almeno una lettera maiuscola". A dark button labeled "Completa registrazione" is at the bottom.

Find Your Book

FindYourBook

< Indietro

Registrazione

Nome

Cognome

Username

Email

Password

Conferma Password

Data di Nascita

Ruolo: ☒ Lettore ☐ Casa Editrice

Mostra password

La password deve contenere:

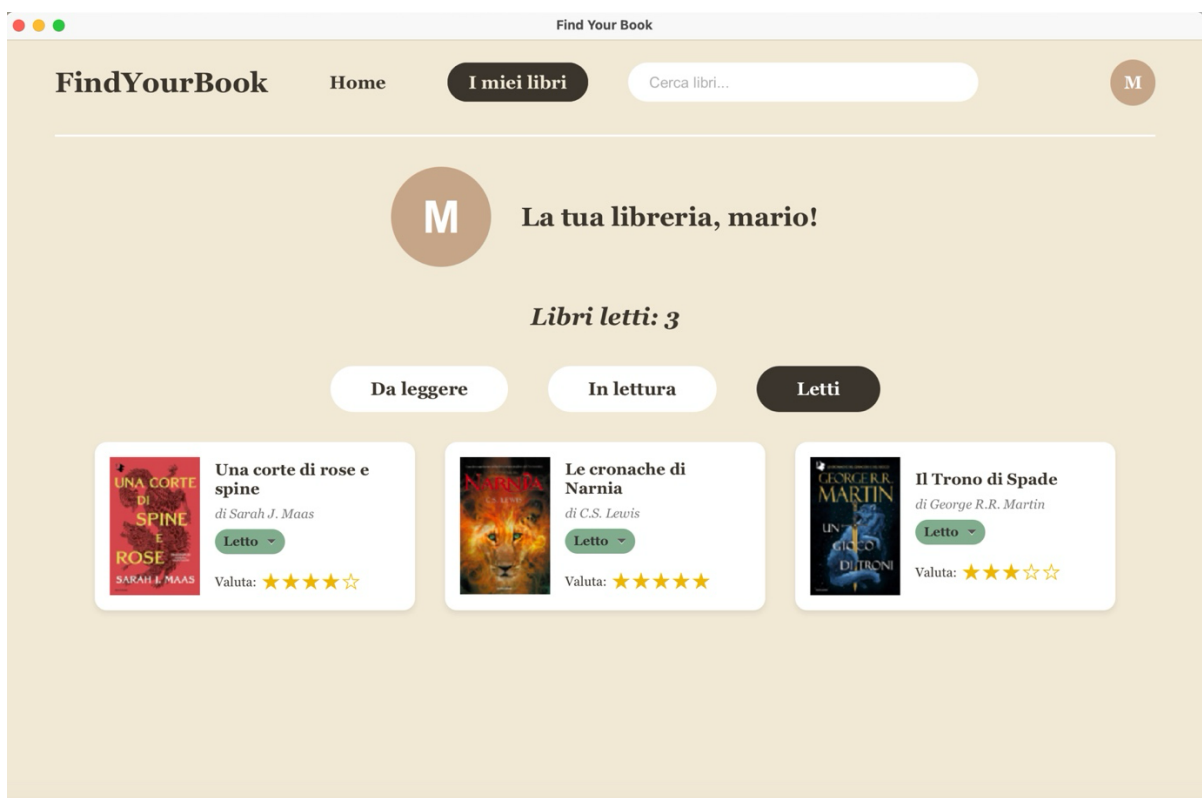
- Almeno 8 caratteri
- Almeno un carattere numerico
- Almeno una lettera maiuscola

Completa registrazione

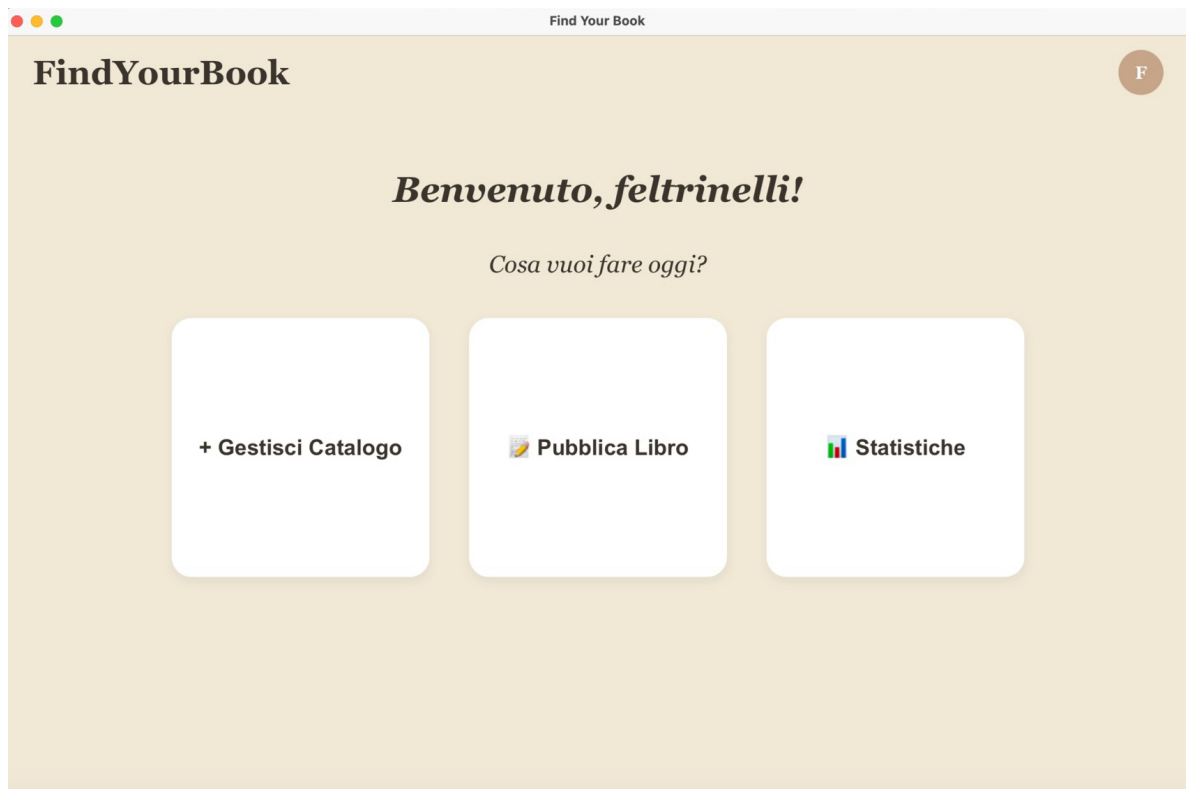
Reader Dashboard



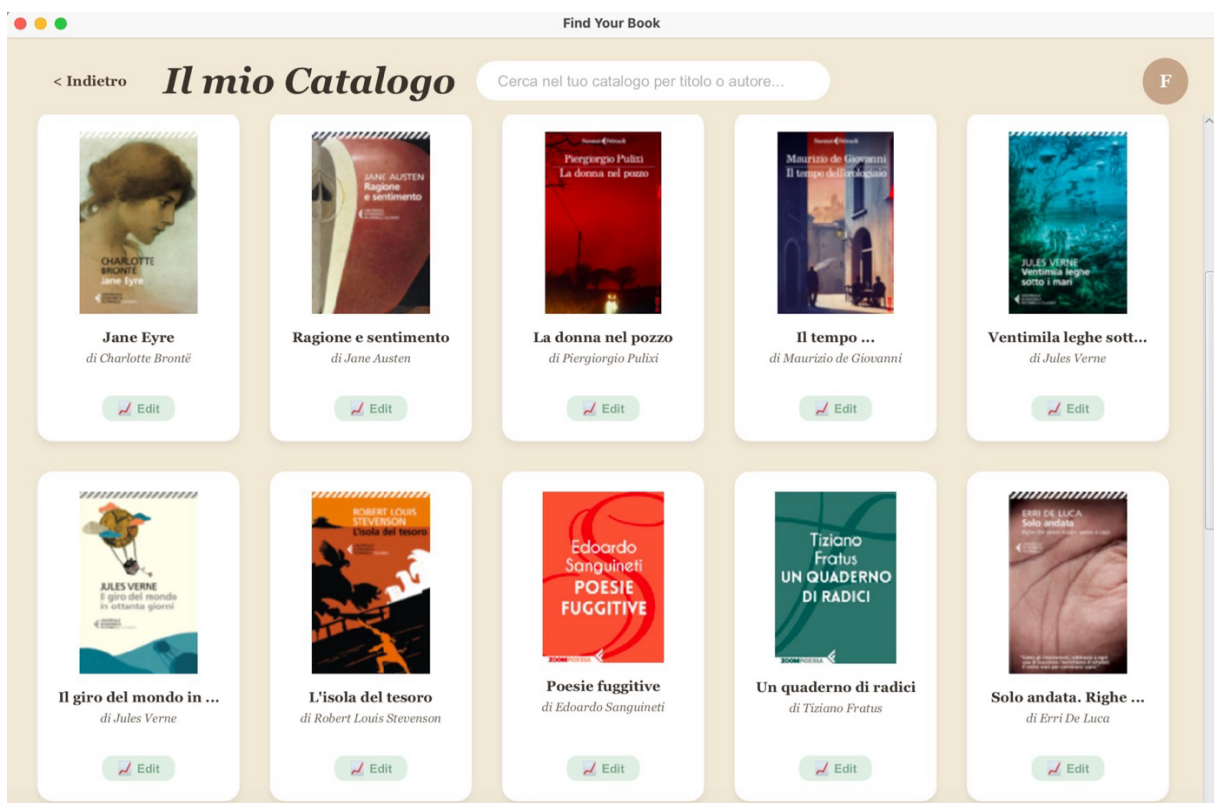
Reader Library



Publisher Dashboard



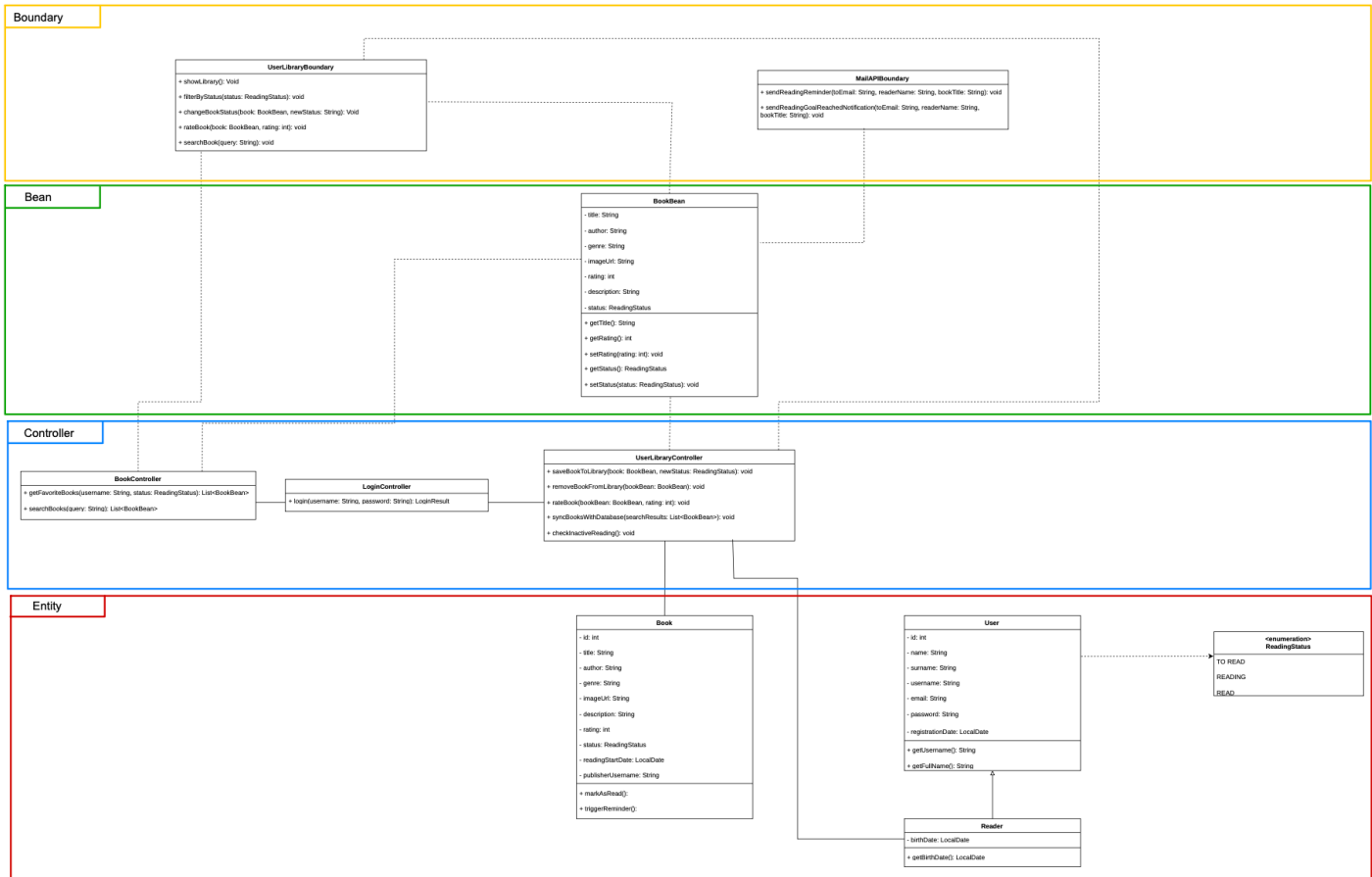
Publisher Catalog



3. Design

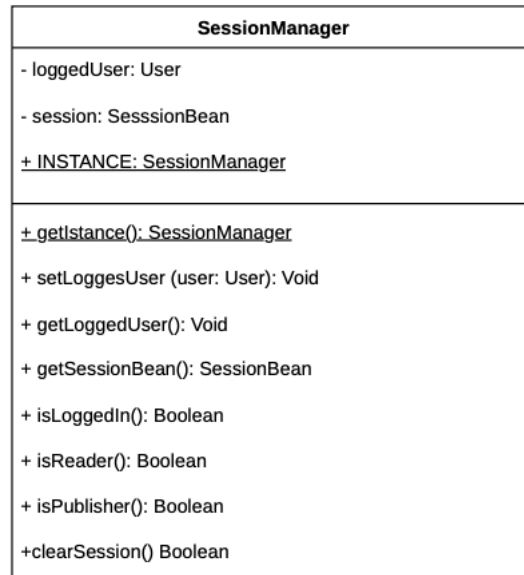
3.1 Class Diagram

3.1.1 VOPC analysis

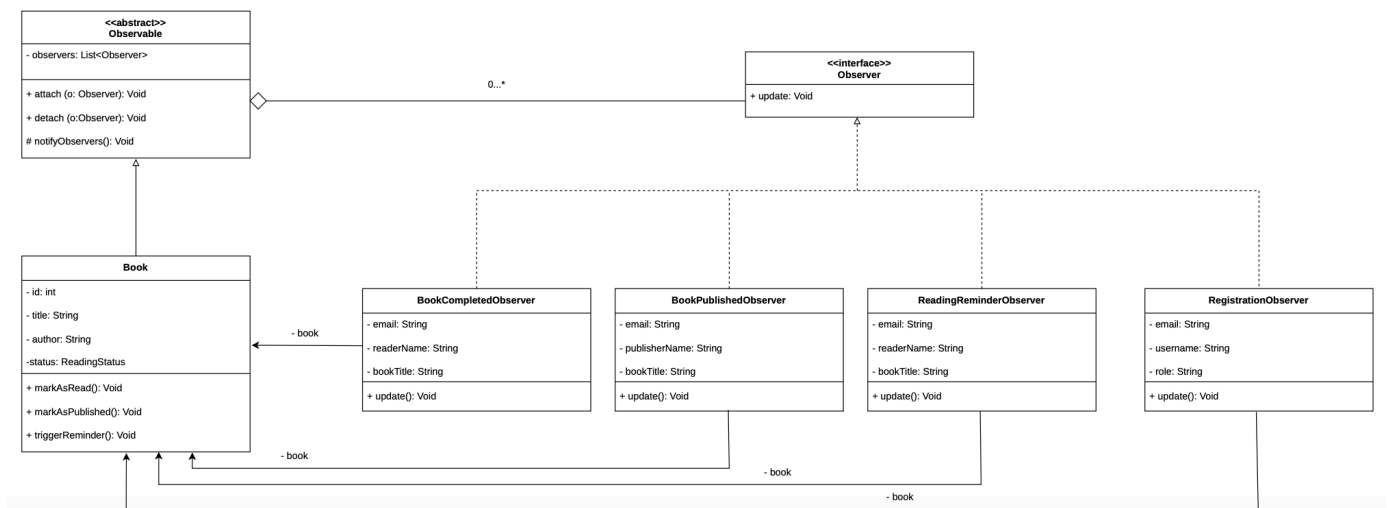


3.1.2 Design-level Diagram

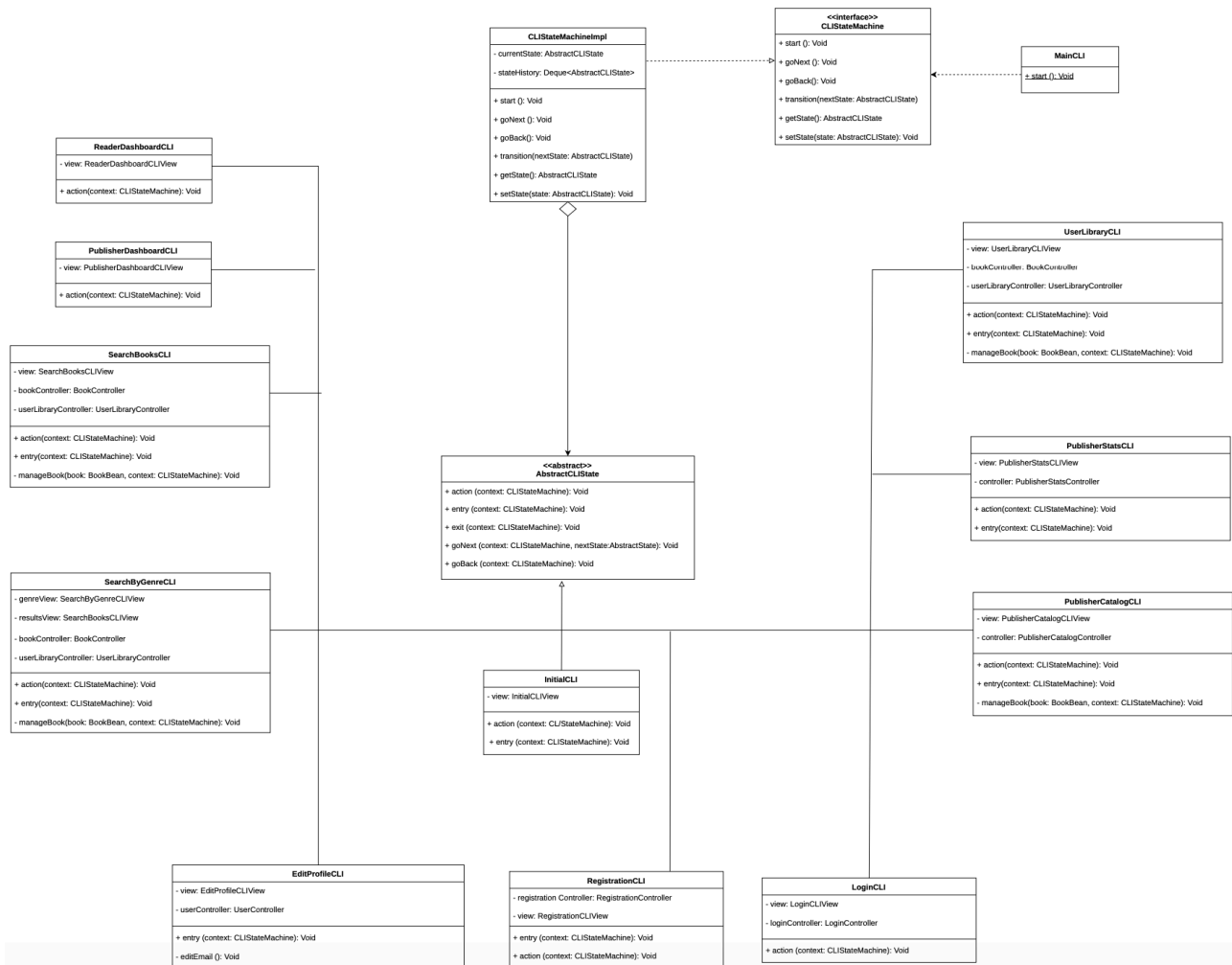
Singleton Pattern:



Observer Pattern:



State Pattern:



3.2 Design Patterns

1. SINGLETON

I adopted the Singleton pattern for `SessionManager` because the application needed a single, shared point to store session data (the currently logged-in user and their role) accessible from any controller without having to pass it explicitly as a parameter. This ensures that the class has exactly one instance throughout the entire application. The implementation manages access control and centralizes authorization logic, exposing methods to check login status and user roles directly from within the various controllers.

2. OBSERVER

I used the Observer pattern to decouple state changes and events (such as book completion, publication, or inactivity reminders) from notification handling. The control and domain classes do not rigidly invoke notification services directly; instead, they rely on registered observers.

BookCompletedObserver: Implements the Observer interface and triggers `NotificationService.sendReadingGoalReachedNotification` inside the `update()` method.

BookPublishedObserver: Implements the Observer interface and triggers `NotificationService.sendBookPublishedNotification` inside the `update()` method.

ReadingReminderObserver: Implements the Observer interface and triggers `NotificationService.sendReadingReminder` inside the `update()` method.

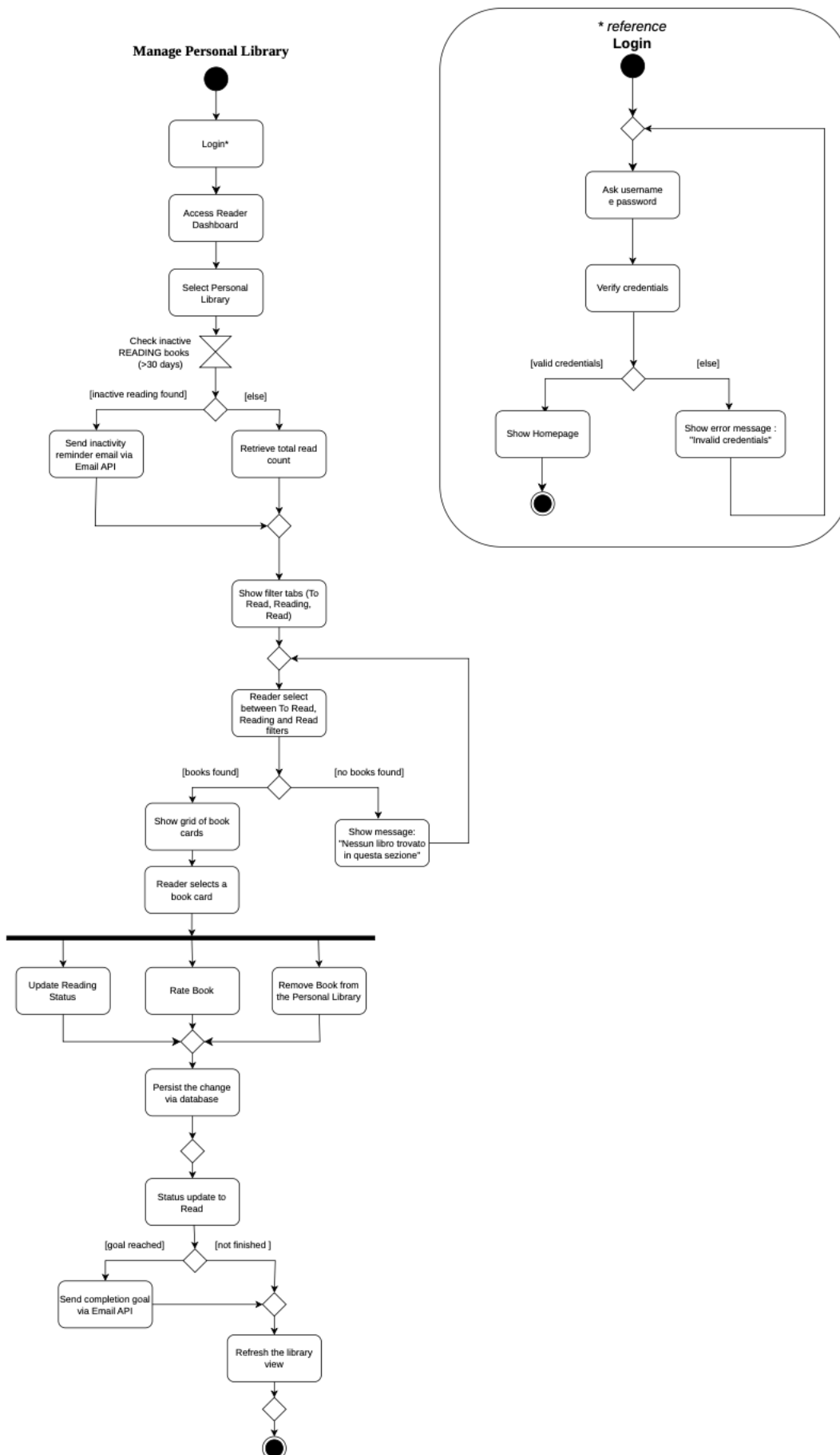
RegistrationObserver: Implements the Observer interface and triggers `NotificationService.sendRegistrationConfirmation` inside the `update()` method.

This design makes the system easily extensible: new notification channels or event types can be added simply by inserting a new observer without modifying existing classes.

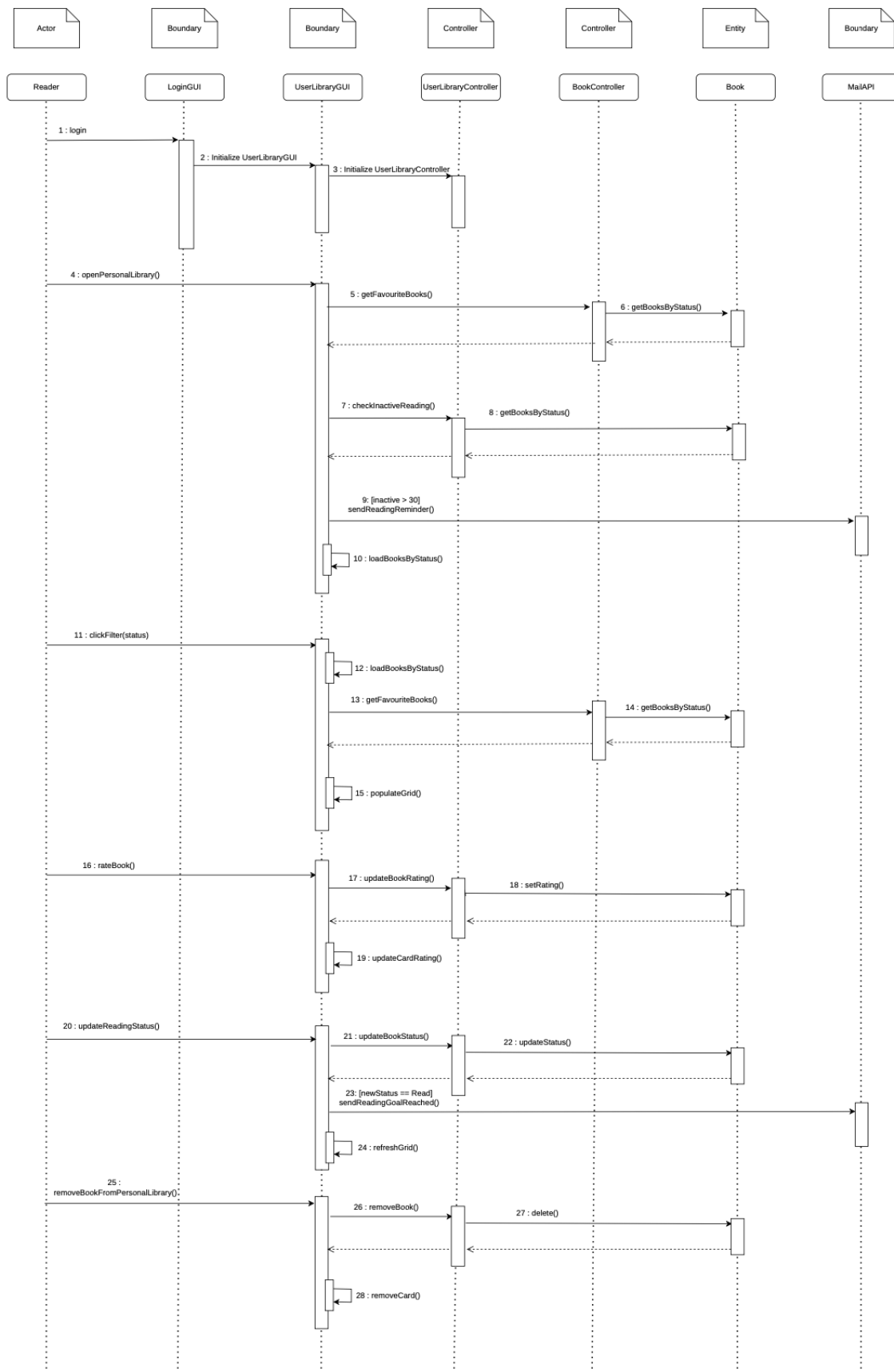
3. STATE

I chose the State pattern to replace a centralized and rigid navigation structure within the command-line interface, delegating screen management to a dedicated state machine. Each screen or screen-functionality becomes an independent `ConcreteState` extending `AbstractCLIState`, defining the lifecycle methods `entry()` for view initialization, `action()` for input handling and transition logic, and `exit()` as an empty default. `MainCLI` acts as the Client by initializing the state machine, while `CLIStateMachineImpl` functions as the Context, storing the current state and a history stack (`Deque<AbstractCLIState>`) for forward and backward navigation via push/pop. Each state manages its own transitions independently within the `action()` method, keeping each screen self-contained and making the system open to extension without altering pre-existing code.

3.3 Activity Diagram

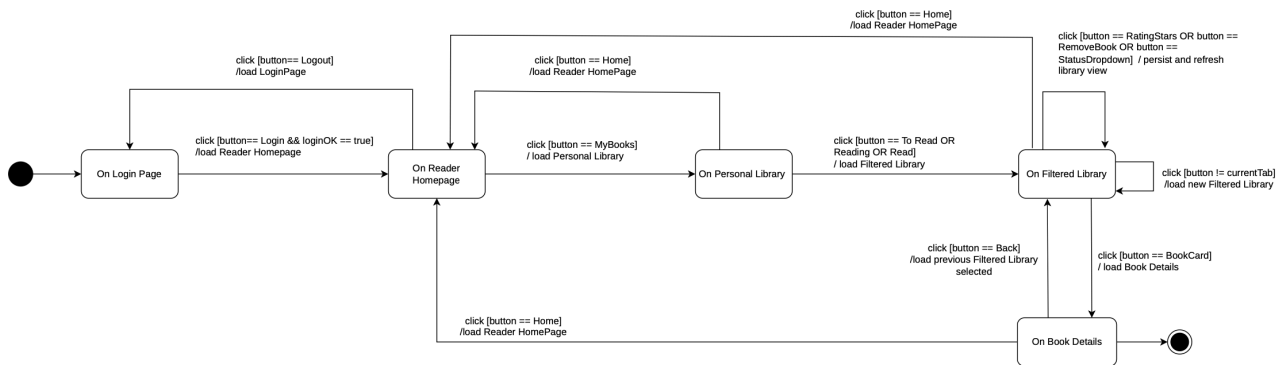


3.4 Sequence Diagram



"Note: The Reader interactions (rating, updating status, and removing a book) are modeled sequentially to illustrate a standard use-case scenario of independent events triggered over time."

3.5 State Diagram



4. Testing

Login with invalid credentials The authentication system was tested by attempting a login with an unregistered user or email address. This test ensures that the system correctly rejects invalid credentials by raising a LoginException.

Rejection of duplicate registrations The system was verified to prevent two accounts from sharing the same username or email address. After a first successful registration, a second attempt using the same username correctly raises a RegistrationException.

Saving and updating the user library The reader's library management was tested in-memory by setting up a mock logged-in reader in the session. This test verifies that saving a book into the user's library with a specific reading status completes successfully without throwing unexpected exceptions.

5. Code

GitHub:

[\[https://github.com/auroracicchetta/FindYourBook\]](https://github.com/auroracicchetta/FindYourBook)

Sonar Qube Cloud:

[\[https://sonarcloud.io/project/overview?id=auroracicchetta_FindYourBook\]](https://sonarcloud.io/project/overview?id=auroracicchetta_FindYourBook)

6. Video

[\[https://youtu.be/OdJB4W01A-I\]](https://youtu.be/OdJB4W01A-I)